Sorting as an Agentic Search Space

Mathematical structure, runtime-aware evaluation, and sampled sorting portfolios

Abstract

Sorting is unusually tractable for agent-guided algorithm research: candidate programs have a crisp ordering contract, exact output verification is cheap, and the input exposes measurable structure such as natural runs, duplicate cardinality, inversions, and positional disorder. Those properties support a closed loop of hypothesis, implementation, oracle check, and held-out benchmark. They do not make comparison count a sufficient performance objective. We synthesize the project's run-adaptive Powersort/Ford-Johnson results with work on learning-augmented sorting and AI-discovered routines, then implement **hybrid_ladder**, a deterministic sampled portfolio that selects among Ford-Johnson block caps using estimated displacement scale. Across fresh input seeds it often reduces comparisons, but at one million elements it is about 1.8× slower than Powersort on random permutations and 2.4× slower than std::sort in the measured run. A later rotation-merge ablation lowers tracked heap from 2.05 to 0.75 MiB at 4% corruption but is slightly slower, exposing another resource tradeoff. Sorting is suited to agentic *search* because correctness and structure are observable, while progress must be scored on target-machine latency, memory limits, and workload robustness rather than a convenient surrogate alone.

MAIN RESULT	The ladder lowers random-input comparisons from 18.5990 to 18.5275 per element, yet measures 135.97 ns/element versus 74.92 for Powersort and 57.64 for std::sort. Less work in the comparison model did not mean less elapsed time.

Research scope

The paper treats an AI agent as a research collaborator that proposes and evaluates algorithm changes. It does not claim that an AI model is part of the runtime sorter, that the new portfolio is production ready, or that this local experiment establishes a universal speed ranking. The algorithm is an empirical candidate and a controlled negative result for comparison-count-only optimization.

1. Why sorting supports agent-guided discovery

A useful agentic benchmark has a short path from proposal to falsifiable evidence. Sorting provides that path, while exposing enough interacting choices to make the search nontrivial.

Exact correctness feedback

For a comparator that defines a strict weak order, a candidate can be checked against a trusted sorted reference. This gives a deterministic pass/fail signal on each generated input, including duplicate keys and stability checks when required. The reference is an oracle for correctness, not a performance objective: a candidate can pass every check and still be slower or use excessive memory.

Visible structure and bounded interventions

The input offers interpretable statistics. Natural ascending/descending runs support entropy-sensitive merge trees; duplicate frequency changes the value of large Ford-Johnson blocks; sampled inversions at a fixed positional lag estimate disorder in a declared displacement model. Each statistic can motivate one bounded change while the remaining implementation stays fixed, enabling controlled ablations.

Multiple objectives are measurable

The lab records exact comparator calls, wall-clock time in a separate uninstrumented mode, peak auxiliary heap, a conservative Ford-Johnson stack bound, and merged span. This makes it possible to detect when an optimization transfers work between dimensions. Hardware counters would add evidence about cycles, instructions, branch misses, and cache events; they were unavailable in this environment because Linux exposes perf_event_paranoid=4.

The right search loop

A reliable agent loop is: (1) state a mechanism and a narrow prediction; (2) implement the smallest reversible variant; (3) verify outputs and comparator accounting; (4) benchmark a declared train grid; (5) freeze thresholds; (6) evaluate fresh seeds and distributions; and (7) compare runtime and resource costs on the target hardware. Sorting is suitable because all seven stages can be automated and audited. Its main trap is proxy optimization: exact comparisons are mathematically elegant, while machine time also depends on movement, branches, allocation, cache locality, compiler output, and frequency scaling.

These properties align with two established research styles. Learning-augmented sorting formalizes how position predictions can reduce comparison complexity as prediction error changes [4]. AlphaDev formulates small-sort routine discovery as a game and optimizes measured latency and code length, with improvements tested across architectures [1]. The current project combines interpretable input signals with an explicit portfolio policy; it does not train a reinforcement-learning model.

2. Mathematical signals and their limits

Decision-tree lower bound.

For n distinct keys, comparison sorting needs at least lg(n!) comparisons in the worst case, because a decision tree must distinguish n! possible orders. Stirling's approximation gives lg(n!)/n = lg n - lg e + o(1). This is a valuable reference for comparison efficiency. It says nothing by itself about cache misses, branch prediction, or the cost of a comparison in the target implementation.

Run entropy.

If natural runs have lengths $L_1,...,L_r$ and $p_i=L_i/n$, their entropy $H=\Sigma p_i \lg(1/p_i)$ expresses how the existing order is distributed. Powersort's merge policy has an entropy-sensitive merge-cost bound $Hn+O(n)$ [2]. This signal is both mathematical and observable: a run scan constructs the decomposition a sorter can exploit. It offers the agent a principled action space around run detection, base-run extension, merge-tree choice, and galloping.

Prediction error and empirical displacement.

In the project's dispX generator, items are ordered by rank plus Gaussian noise of standard deviation σ. For a pair separated by positional distance d, the model motivates an inversion probability near $\Phi(-d/(\sqrt{2}\sigma))$. A sampled inversion rate can therefore classify scales within this generator family. It is only a predictor for this family: it does not observe per-item predicted positions, and the local C++ algorithms do not satisfy or test the theorem of Bai and Coester [4]. A fixed sample is also vulnerable to adversarial construction.

3. Prior work in this laboratory

The repository has already explored complementary points on the comparison/runtime frontier. These findings constrain what a new agent proposal should target.

Run-adaptive Ford-Johnson base cases.

powersort_fj retains Powersort's run targets, merge policy, and galloping, using Ford-Johnson only when a detected prefix does not make binary insertion preferable. On the original n=1,000,000 random grid, prefix reuse brought it to 18.580018 comparisons per element, saving 0.019021 versus Powersort. It had no regressions in 81 paired count cases in the committed milestone grid, but its runtime advantage was not established; later timing showed mixed per-seed effects.

Large blocks and the sampled binary gate.

The auto-2048 portfolio reached 18.520456 comparisons per element on random input, close to the finite-size lower bound 18.488885, while its run-adaptive robustness was poor: structured-input regressions reached +6.597 comparisons per element and median random runtime was about 96% slower than Powersort in the earlier study. hybrid_gate uses adjacent/equality probes and inversion fractions at distances 64, 256, and 1024 to choose between PFJ and auto-2048. On the original grid it captured most of the random comparison saving with much smaller measured comparison downside; this still did not make it a fast choice in the present runtime study.

Other structural discoveries.

The same repository implements directional entropy-sensitive mergesort, a Pingpong Powersort data-movement variant, and a learned Shellsort gap sequence. Together these candidates illustrate distinct agent search levers: merge schedule, scratch-buffer choreography, base-case method, and a finite parameter sequence. Their local results are distribution-dependent. The strongest research pattern is not that one algorithm wins everywhere; it is that sorting allows clean hypotheses about structure to be carried through exact implementation and counterexample-rich evaluation.

The new question

Can a multi-cap sampler exploit the displacement-scale crossover more finely than a binary gate while preserving a low-regret fallback? The design uses prior seeds to set tiers and reserves fresh-seed counts for evaluation. Because earlier evidence showed duplicate cardinality also changes the winning cap, the ladder reuses the pre-registered K≈112 duplicate veto. This is a comparison-aware design hypothesis; the following runtime results test whether that is the right objective.

4. New candidate: hybrid_ladder

For n<131,072 the algorithm delegates directly to powersort_fj. At larger n it samples adjacent pairs to reject highly ordered inputs and estimate duplicate density. If the estimate suggests K≤112, it falls back to PFJ. Otherwise, it samples pairs separated by 256 positions and converts the inversion fraction to a per-thousand rate r.

Condition	Branch
Duplicate/adjacent veto, or r < 20	powersort_fj
20 ≤ r < 160	hybrid_fjauto (cap 128)
160 ≤ r < 300	hybrid_fjauto512
300 ≤ r < 400	hybrid_fjauto1024
r ≥ 400	hybrid_fjauto2048

The tiers approximate inversion rates from the Gaussian displacement model at σ values associated with the observed cap crossovers. They were selected from the existing displacement and duplicate-cardinality studies, then evaluated on fresh seeds 4–6 and off-grid σ values. The thresholds are empirical, deterministic, and easy for an agent to mutate; they are not a theorem or an adversarial guarantee. The sample examines up to 4,096 lagged pairs plus up to 2,048 adjacent positions at n=1,000,000. The selected branches are unstable, as are the Ford-Johnson hybrids.

5. Experimental protocol

Implementation and correctness.

C++20, GCC 15.2, -O3 -march=native, on an 11th Gen Intel Core i7-11800H at 2.30 GHz (16 logical CPUs), Linux x86_64. The registered hybrid_ladder passed the full repository self-test; the new dispatch/accounting cases check characteristic input families, output equality, and that total comparator calls equal probe calls plus the selected branch. Build identifier: ladder-v1.

Count experiment.

The predeclared train grid contains 336 count rows at n=1,000,000: 8 algorithms × 14 input distributions × seeds 1–3. The held-out grid contains 240 rows: 8 algorithms × 10 distributions × seeds 4–6, including off-grid displacement scales and duplicate cardinalities. Every row has ok=1. Count-mode times are instrumentation-contaminated and are excluded from all runtime claims.

Runtime experiment.

A separate uninstrumented profile records seven serial repetitions per algorithm/input for six algorithms on five n=1,000,000 inputs. The focused comparison includes std::sort, Powersort, PFJ, the old gate, the new ladder, and auto-2048. Medians are reported in ns per element. Runs were not CPU-pinned or interleaved by algorithm. Frequency scaling and the short single-host study limit small differences; treat the broad gaps as local measurements, not portable rankings. Linux performance counters were blocked by the host security setting, so no branch-miss or cache-miss claims are made.

Input families.

The count grid spans random permutations, duplicates (K=16/96/256), sorted/run-structured and nearly sorted data, organ-pipe and tail patterns, plus dispX at σ=64 to 2,048. Held-out rows add K=48/192 and σ=96/192/384/768/1536/3072. Inputs are generated reproducibly from fixed seeds; only the dispX family directly matches the Gaussian model used to set inversion tiers.

6. Results: surrogate wins, runtime loses

Comparison results.

On random permutations, the ladder uses 18.527490 comparisons per element, compared with 18.599039 for Powersort, 18.580018 for PFJ, and 18.520456 for auto-2048. On held-out random seeds it uses 18.527617, compared with 18.579358 for PFJ and 18.520597 for auto-2048. Across the 30 held-out algorithm/distribution/seed pairs shared with PFJ, the ladder uses fewer comparisons in 23 cases; the worst observed regression is +0.1831 per element, on K=96 duplicates, near the measured branch crossover. This is a fresh-seed result, not an independent machine or workload replication.

Random-input resource measures.

Powersort and PFJ both peak at about 4.00 MB of tracked heap and have merge span 13.968 elements per input element. The ladder's random path reaches the same 4.00 MB heap and 8.999 merge span as auto-2048; its conservative FJ stack bound is 273,272 bytes, versus 6,204 for PFJ. These measures help explain why a lower comparison count alone does not decide system cost.

Runtime across input families

Median uninstrumented runtime (ns/element; seven serial repetitions)

Input	std::sort	Power	PFJ	Gate	Ladder	Auto-2048
Random	57.64	74.92	80.54	159.91	135.97	136.11
Sorted	8.25	0.44	0.73	0.48	0.74	0.46
Nearly	9.79	9.65	11.58	10.27	10.33	23.43
disp1024	38.25	47.04	53.11	107.53	109.17	108.67
dup16	20.72	42.44	43.02	51.14	43.08	103.30

Table 1. One local host, n=1,000,000, one generated seed per distribution. The ranking varies by workload; no small differences should be generalized beyond this setup.

The largest gaps are on random and displaced inputs: the ladder is close to auto-2048 but substantially slower than both Powersort and std::sort. On nearly sorted input, its adjacent-probe veto retains an adaptive branch; this keeps its time near Powersort but does not surpass std::sort in this sample. The data support target-machine timing as the decision criterion and treat comparison count as a diagnostic covariate.

7. Interpretation and next experiments

The ladder demonstrates a useful agentic iteration and a useful rejection. A mathematically motivated sample can predict which block size reduces comparisons over a known input family, and the exact comparator/output oracle makes the behavior easy to validate. Fresh seeds preserve the direction of the random comparison result. Yet the runtime data show the search policy is not a speed improvement: random median is 135.97 ns/element versus 74.92 for Powersort and 57.64 for std::sort. The older binary gate is also slow (159.91 ns/element). Even where the count model predicts a useful branch, the branch's implementation work and sampling cost dominate elapsed time.

This result is consistent with established work showing that comparison and swap counts do not fully explain sorting runtime: pipeline, branch behavior, and memory hierarchy can change which design is faster [3]. AI discovery work makes the complementary methodological point: optimize and validate latency at the machine-code/runtime level, not solely an algorithmic proxy [1]. The appropriate objective is a constrained vector, such as (median latency, tail latency, peak memory, stability/correctness), with workload weights declared in advance. Comparisons and merge span remain diagnostic features, not the score.

What would improve the next agent loop

First, add repeated, interleaved timing and CPU affinity or frequency controls, and obtain permission to collect hardware counters. Second, train the policy on runtime outcomes from the exact target compiler and CPU, using comparison features only as model inputs. Third, report per-distribution regret and a conservative fallback: a candidate should be accepted only when its measured latency gain exceeds noise while its worst-case workload loss stays inside a declared bound. Fourth, consider replacing Ford-Johnson's bookkeeping or evaluating a branch-light base sorter, since prior studies already identify FJ overhead as the comparison-versus-runtime bottleneck. Finally, test multiple key types and record code size, allocations, bytes moved, and stability requirements.

Conclusion

Sorting is suitable for AI-guided algorithm search because its contracts are exact, its latent structure has mathematical descriptions, and experiments can expose many counterexamples cheaply. The new ladder validates a richer displacement policy on held-out seeds and lowers comparison counts, but loses on the practical runtime objective. The strongest finding is therefore methodological: mathematical structure supplies hypotheses and interpretable features, while target-machine time and workload robustness must decide whether an algorithm is better.

8. Follow-up: runtime portfolio from run density

The comparison-aware ladder failed its runtime objective, so this follow-up changes the decision signal instead of further tuning Ford-Johnson block sizes. Powersort is fast when the input has few long ascending or descending runs; std::sort is faster on high-disorder arrays in this GCC/Intel setup. Natural-run count is a meaningful feature, while the position of local runs distinguishes a mostly sorted prefix with a noisy tail from uniform disorder.

hybrid_run_gate samples 128 adjacent pairs in each of 16 equal positional bins. A bin is nearly monotone if either its ascending or descending sample rate is at most 2%. If at least 13 bins meet the condition, it selects Powersort; otherwise it selects std::sort. Stratification lets it route a sorted prefix with a random tail to Powersort because most bins remain ordered. The rule adds 2,048 to 4,096 comparisons at n=1,000,000 and has no worst-case guarantee.

The strict build and full self-test passed. Across 78 count cases (13 inputs, seeds 1-6), every output passed the reference check and each gate count equaled its selected branch plus measured probe cost. The primary timing study used five seeds and 21 serial repetitions per seed. Timed processes were pinned to CPU 2 and algorithms were interleaved by input with a rotating first algorithm. The host governor remained powersave, so CPU frequency still varied.

Median runtime by input (ns per element)

Input	std::sort	Powersort	Run gate
Random	52.55	74.76	52.90
dup16	24.86	51.27	23.78
Sorted	7.50	0.54	0.54
Reversed	5.31	0.67	0.75
runs32	49.17	60.76	49.56
runs1024	43.33	42.67	43.09
Nearly1	8.80	8.97	8.78
tail10	24.32	7.91	7.99
Organpipe	68.51	2.12	2.16
disp64	31.15	38.78	30.25

Table 2. Each cell is the median of five per-seed medians; each uses 21 uninstrumented serial repetitions. Inputs have n=1,000,000. The timing data also include displacement 256 and 1024, omitted here for space.

Across 50 input/seed pairs, median excess over the faster standalone branch was 0.96% (mean 0.81%). The gate stayed within 5% of that branch in 42 pairs and within 1% in 26; its worst measured excess was 23.99%, concentrated in very fast monotone cases where relative timer noise grows. This suggests a small structural sample can route close to the locally faster algorithm on the declared suite. It does not establish a universal speedup: the gate adds probes and small differences remain sensitive to CPU frequency.

The follow-up sharpens the agentic research loop. Exact verification and mathematical run structure make it practical to propose a routing rule, expose every threshold, and measure regret against both branches. The first portfolio optimized comparison count and lost on runtime; the second targets runtime directly and stays near the measured best branch on most tested cases. Next, repeat on a different CPU and compiler and add hardware counters if the host allows them.

9. Follow-up: inversion-sensitive insertion

A second presortedness measure is the inversion count I, the number of out-of-order index pairs. It complements natural-run structure: insertion work follows local displacement, while Powersort exploits monotone runs. Theory studies sorting adaptive to I [10, 11], but an inversion feature still needs a target-machine test because comparisons, element movement, and cache behavior have different costs.

inversion_binsert checks each adjacent pair and skips ordered elements. For a descent, it exponentially searches backward, binary-searches the stable upper-bound insertion point, and shifts the intervening range. Every shifted element removes one inversion, so the total shifts equal I. A budget of 16n shifted elements triggers stable-sort fallback; this protects against quadratic transport when the classifier is wrong while keeping the sorter stable.

hybrid_inversion_gate samples 256 adjacent pairs. If none are descents it calls Powersort directly. Otherwise it samples 2,048 pairs at index distance 64: at most eight inversions route to inversion_binsert; a higher rate delegates to the run-density gate. This lagged-inversion rate is a useful feature on Gaussian rank perturbations, not a distribution-free estimate of total I. The movement budget, exact output oracle, and held-out tests make the empirical rule falsifiable.

The strict build and full self-test passed, including stable-order checks and nine exact gate-accounting cases. The 1m grid covers eight distributions and five seeds; the 200k follow-up uses fresh seeds and unrelated structured inputs. Timings were interleaved and pinned to one logical CPU, with the governor left in powersave mode.

Median runtime by input (ns per element, n=1,000,000)

On the 1m count grid, the gate selected insertion in 9 of 24 cases, Powersort directly in 3, and the run-density portfolio in 12.

Input	std::sort	Powersort	Run gate	Inversion insertion	Inversion gate
sorted	7.70	0.53	0.73	0.68	0.57
disp4	17.43	17.58	16.99	11.73	11.70
disp8	18.45	21.26	19.03	14.91	14.82
disp16	21.32	25.94	21.31	19.44	19.91
disp32	23.60	30.03	23.71	33.07	23.30
disp64	25.82	34.46	25.96	35.65	26.49
random	52.99	76.96	53.17	66.69	53.58
tail10	25.05	8.27	8.38	17.09	8.31

Table 3. Median of five per-seed medians; each seed uses 21 uninstrumented serial repetitions. Processes were pinned to CPU 2. The host governor was powersave and CPU frequency was not fixed.

On displacement scales 4, 8, and 16, inversion insertion is faster than both broad standalone baselines; the gate captures most of that gain. It returns to the run-density policy by scale 32. Across 40 input/seed pairs at n=1m, regret against the fastest standalone sorter had median 0.63%, mean 1.84%, and stayed within 5% in 32 pairs. On the independent 200k grid, it selected insertion in 8 of 65 count cases; median regret across 65 time pairs was 0.81% (mean 2.92%) and 54 stayed within 5%. Large relative outliers occur on very fast monotone inputs where absolute timer differences are small. These local results do not establish a universal speedup; hardware counters were unavailable.

The agentic search loop now has a third interpretable signal: inversions complement run density and positional displacement. Future work should validate the movement budget on other data types and CPUs, measure memory traffic directly, and expand low-disorder generators beyond Gaussian rank noise.

Modern sorting work supports measuring latency: AlphaDev optimized latency and evaluated across 100 machines [1], while Quicksort analysis shows branch and pipeline effects alter comparison costs [3]. Treat correctness, stability, memory, and fallback limits as constraints; score controlled latency and held-out workload regret; retain comparisons, inversions, and runs as explanatory features. Hardware counters could explain timing, but were unavailable here.

10. Follow-up: distance to monotonicity

The next structural parameter is k = n - LNDS(a): the fewest deletions that leave a nondecreasing subsequence. Distance to monotonicity is an established LIS measure [12]. It distinguishes a small number of misplaced extreme keys from inputs with many inversions or broad displacement. In outpctP, P percent of positions receive large keys in random order; deleting them restores the sorted backbone.

deletion_backbone_sort scans a nondecreasing backbone. Whenever the next value falls below its top, it removes both values into an outlier buffer. The removed pairs are disjoint inversions, so every nondecreasing subsequence must drop at least one value per pair. Thus q removed values satisfy q <= 2k. If q stays within n/16, the algorithm sorts those values and merges them backward in O(n + q log(q+1)) work using O(q) extra storage. It restores the prefix and falls back to std::sort when the budget is exceeded. The algorithm is unstable.

The one-million-element table reports median ns per element on the density training grid. The comparison column is a diagnostic; latency decides the result.

Outliers	Powersort	Inv. gate	Backbone	Cmp/elem
outpct1	6.83	6.90	5.59	2.30
outpct2	12.55	54.17	6.05	2.65
outpct3	17.51	48.45	6.91	3.03
outpct4	19.59	49.75	47.81	28.27

Table 4. Median of three seed-level medians; each seed uses 21 interleaved serial repetitions. CPU 2 pinned, powersave governor, frequency variable.

For 1%, 2%, and 3% corruption, the candidate beats the fastest standalone baseline in all 18 training pairs across n=200k/1m, with median regret -52.16%. Fresh seeds 7--9 reproduce the win in all 18 pairs, median regret -49.85%. At 4%, the fixed outlier budget often falls back and loses to Powersort. The retrying displacement-window heap is rejected here: at 1m and 1% outliers it takes 676.96 ns/element, versus 5.59 for deletion-backbone. These are local results on generated sparse-corruption inputs, not a universal sorting improvement.

This candidate turns a known monotonicity-distance measure into an output-verifiable sorting mechanism. Its disjoint-pair certificate explains why small k guarantees a small repair set; the fallback and held-out seeds test the cost when the premise weakens. Section 12 reports a follow-on ablation that preserves the sorted backbone after overflow and merges it with the untouched suffix.

11. Modern objective and overflow-routing ablation

Comparison count remains useful for decision-tree lower bounds and can dominate when comparisons are expensive, such as string or user-defined comparators. For primitive in-memory keys, it is only one component of cost: branch behavior, cache transfers, data movement, allocations, compiler output, and frequency scaling can change the latency ranking. BlockQuicksort reduces branches dependent on comparison outcomes [13]; IPS4o evaluates branch prediction, cache efficiency, and memory transfers alongside work [14]. We therefore treat correctness, required stability, and memory limits as constraints, use uninstrumented target-machine latency and held-out workload regret as the primary outcomes, and retain comparisons and tracked allocation as explanatory metrics. Hardware counters are unavailable on this host, so we make no causal branch/cache claims.

The overflow router tests a fixed signal rather than comparison count: on budget overflow, use Powersort when scan progress is at least n/4, otherwise use std::sort. The restored prefix remains an exact permutation, so both branches preserve correctness. This is a portfolio hypothesis, not a consequence of the q <= 2k certificate.

Outliers	Old fallback	Routed	Best baseline	Cmp/elem	Heap MiB
1%	2.80	2.94	4.92	2.35	0.38
2%	4.17	4.12	7.17	2.70	0.75
3%	4.86	4.84	10.76	3.10	0.75
4%	27.99	9.82	13.83	3.42	2.38
6%	34.66	12.74	15.73	4.40	2.73
8%	35.59	17.33	19.76	5.11	1.51
10%	34.63	17.93	21.97	5.99	2.01

Table 5. One million elements; three fresh seeds (21--23), each timed in 21 interleaved serial repetitions. Times are ns/element. The best baseline is the minimum among std::sort, Powersort, inversion insertion, and inversion gate. Memory is peak tracked heap from separate count runs.

At 4--10% outliers, the router beats the original std::sort fallback in all 42 size/seed cases, with median paired change -52.63%. Its median regret versus the fastest baseline is -14.13% (mean -15.78%; 95th percentile 0.68%). The original 1--3% path changes by a median -0.87%, preserving its speed while showing no consistent change. At 4% density, the routed version uses about 3.42 comparisons/item and 9.82 ns/element, versus Powersort's 4.42 and 13.83; comparison reduction and elapsed time align in this case, but the count alone does not explain the memory premium of roughly 2.4 MiB on the Powersort fallback.

The route is specialized: on one-million-element controls it is about 72 ns/element on random input (near std::sort), but is slower than Powersort on tail10 and slower than std::sort on displacement-16. The correct conclusion is a bounded workload-specific win under a measured latency objective, with memory cost exposed as a tradeoff. Section 12 measures a preservation variant on fresh seeds.

12. Preserving completed work after overflow

The overflow scan maintains a sorted backbone of length b and q extracted values when it reaches index m = read + 1; its invariant is b + q = m. The preserve variant sorts the extracted values, merges them into the backbone to complete a sorted prefix, runs Powersort on only the untouched suffix of length r = n - m, then merges the two sorted ranges. Before overflow, the disjoint inversion certificate still implies q <= 2k for k = n - LNDS(a). The proof explains why a small deletion distance bounds repair items, while the measured overflow index decides whether this continuation is fast.

The last merge uses std::inplace_merge. The C++ working draft specifies at most N-1 comparisons when enough additional memory is available, and O(N log N) comparisons otherwise [15]. This is a real algorithmic tradeoff: the implementation may spend several MiB of temporary storage to keep merge work linear.

Outliers	std::sort	Powersort	Original	Route v2	Preserve v3	Heap MiB
1%	48.18	4.42	3.18	3.16	3.02	0.38
2%	32.81	7.34	4.05	4.24	3.96	0.75
3%	28.20	9.73	5.10	4.65	5.30	0.75
4%	28.39	11.98	26.47	8.67	6.65	2.02
5%	32.71	15.19	29.75	11.95	10.08	3.24
6%	31.48	15.45	34.19	13.41	12.81	4.03
7%	34.59	18.18	27.70	13.66	14.28	4.03
8%	38.49	19.49	37.31	16.67	17.95	3.60
9%	33.14	17.67	38.24	17.37	17.11	3.28
10%	33.94	18.53	32.46	19.28	17.25	3.01

Table 6. One million elements, fresh seeds 31--33. Latencies are medians of 21 interleaved serial repetitions, aggregated as three-seed medians; heap is the median peak tracked allocation from separate count runs.

At 4--10% outliers, preserve v3 beats route v2 in 32 of 42 size/seed cases, with median paired change -9.15%. Regret against the fastest of std::sort, Powersort, inversion insertion, and inversion gate is -22.96% at the median, -21.38% on average, and -0.25% at the 95th percentile. For 1--3% inputs, median change versus route v2 is +0.67% across 18 pairs.

At 4% density, preserve v3 is faster than route v2 (6.65 versus 8.67 ns/element) despite using 3.75 comparisons per element versus 3.41. This is a direct example where the lower comparison count is not the faster choice. The cost is heap: preserve v3 peaks at 4.03 MiB at 6%, versus 2.78 MiB for route v2 on the same seeds. The preserve candidate also remains uncompetitive on some ordinary controls, so it is a specialized continuation, not a universal replacement.

13. Rotation merge: memory versus measured work

The rotation ablation replaces the final buffered inplace_merge with a symmetric binary-partition merge: split the longer run, find the matching lower/upper bound in the other run, rotate the middle blocks, and recurse. The merge itself needs O(log n) recursion space and no O(n) merge buffer. With a conservative analysis, it uses O(n log n) comparisons and element transfers; this simpler implementation does not claim Recmerge's optimized bounds [16]. The full sorter still allocates the outlier vector and Powersort may allocate suffix workspace, so the measured quantity is peak heap for the complete candidate, not a zero-memory guarantee.

The cost model predicts a trade: buffered merging can move each element a small number of times with linear comparisons when scratch storage is available; rotation merge reduces the final merge allocation but may rotate elements across logarithmically many levels. This is one reason comparisons alone cannot rank the implementations. We compare against the buffered preserve variant rerun on the same fresh seeds (41--43), sizes, and workload grid.

Outliers	Buffered time	Rotate time	Buffered cmp/e	Rotate cmp/e	Buffered heap	Rotate heap
1%	6.52	6.49	2.32	2.32	0.38	0.38
2%	8.06	8.23	2.70	2.70	0.75	0.75
3%	9.67	10.06	3.08	3.08	0.75	0.75
4%	14.41	14.69	3.82	3.64	2.05	0.75
6%	21.72	22.67	5.05	4.66	4.03	0.75
8%	28.33	29.12	5.90	5.05	3.61	0.75
10%	18.83	21.13	6.59	5.78	3.00	0.76

Table 7. n=1,000,000; medians across three seed-level medians. Time is ns/element from 21 interleaved uninstrumented repetitions; comparison and peak-heap figures come from separate instrumented count runs.

Across paired 4--10% corruption cases (n=200k and 1m, seeds 41--43), rotation merge changes latency by a median +4.37% versus buffered preserve and wins 4 of 42 cases. Its tracked heap at n=1m stays near 0.75 MiB across this density range, while buffered preserve uses roughly 2.05--4.03 MiB. Against the fastest standalone baseline, rotation has median regret -13.82% and a 95th-percentile regret of +10.49%. Thus it offers an allocation reduction for memory-constrained settings, not a general latency improvement.

At 4% and n=1m, rotation uses 3.64 comparisons per item versus 3.82 for buffered preserve, yet takes 14.69 versus 14.41 ns/element. Peak heap falls from 2.05 to 0.75 MiB. The count inversion is consistent with additional transfers, but assignments and hardware misses were not measured; it is evidence of the ranking, not a causal diagnosis. The practical policy is to select using latency under a hard memory budget, and use comparison counts to describe algorithmic work or comparator-heavy workloads.

The rotation merge passed the full sorting self-test and 7,203 direct duplicate-heavy merge comparisons against std::inplace_merge. All 15,840 fresh-grid rows passed output validation. The benchmark was pinned to CPU 2, but frequency varied under the powersave governor; performance counters were blocked. Results are local to this compiler, CPU, key type, and synthetic workload.

Reproducibility

Run ./build/sortlab selftest. Reproduce inversion studies with profiles inversion-adaptivity, inversion-heldout, and inversion-scale. Reproduce deletion-distance profiles with profiles deletion-backbone-density and deletion-backbone-density-heldout. Overflow variants use deletion-backbone-route-v2 (seeds 21--23), deletion-backbone-preserve-v3 (seeds 31--33), and deletion-backbone-rotate-v4 (seeds 41--43), each with 21 timing repetitions on CPU 2. Earlier ladder and run-gate artifacts remain in the repository.

References

[1] D. J. Mankowitz et al. “Faster sorting algorithms discovered using deep reinforcement learning.” *Nature* 618, 257-263 (2023). doi:10.1038/s41586-023-06004-9.

[2] J. I. Munro and S. Wild. “Nearly-Optimal Mergesorts.” In *Proc. ESA 2018*, Article 63. doi:10.4230/LIPIcs.ESA.2018.63.

[3] C. Martínez, M. E. Nebel, and S. Wild. “Analysis of Branch Misses in Quicksort.” arXiv:1411.2059.

[4] X. Bai and C. Coester. “Sorting with Predictions.” *Advances in Neural Information Processing Systems 36* (NeurIPS 2023), 26563-26584. NeurIPS proceedings.

[5] L. R. Ford Jr. and S. M. Johnson. “A Tournament Problem.” *American Mathematical Monthly* 66(5), 387-389 (1959). doi:10.2307/2308750.

[6] B. Jin and A. Z. Xu. “Straightforward Entropy-Sensitive Mergesort.” arXiv:2608.10421 (2026).

[7] B. Liu. “A New Gap Sequence for Shellsort: RL-Driven Algorithm Discovery Beyond N^(4/3).” arXiv:2609.29881 (2026).

[8] A. H. Jalilvand et al. “Sorting it out in Hardware: A State-of-the-Art Survey.” arXiv:2310.07903 (2023).

[9] Ordering Research Lab. “Comparison sorting and adaptive merging: research notes and benchmark data.” Local C++20 study, builds ladder-v1, rungate-v1, inv-v4, backbone-v1, backbone-route-v2, backbone-preserve-v3, and backbone-rotate-v4; raw CSVs and source are included with this paper in the repository.

[10] A. Elmasry. “Counting Inversions Adaptively.” arXiv:1503.01192 (2015).

[11] A. Elmasry and A. Hammad. “Inversion-Sensitive Sorting Algorithms in Practice.” *ACM Journal of Experimental Algorithmics* 13, 1.11 (2008). doi:10.1145/1412228.1455267.

[12] M. Saks and C. Seshadhri. “Space Efficient Streaming Algorithms for the Distance to Monotonicity and Asymmetric Edit Distance.” In *Proc. SODA 2013*, 1698-1709. doi:10.1137/1.9781611973105.122.

[13] S. Edelkamp and A. Weiß. “BlockQuicksort: Avoiding Branch Mispredictions in Quicksort.” *ACM Journal of Experimental Algorithmics* 24(1), 1.4 (2019). doi:10.1145/3274660.

[14] M. Axtmann, S. Witt, D. Ferizovic, and P. Sanders. “In-place Parallel Super Scalar Samplesort (IPS^4o).” In *Proc. ESA 2017*, LIPIcs 87, Article 9. doi:10.4230/LIPIcs.ESA.2017.9.

[15] ISO/IEC JTC1/SC22/WG21. “C++ Working Draft, [alg.merge].” eel.is/c++draft/alg.merge.

[16] Z. Dudzinski and A. Dydek. “A Stable Minimum Storage Merging Algorithm.” *Information Processing Letters* 12(1), 1981. doi:10.1016/0020-0190(81)90065-X.

Data artifacts include ladder_v1_counts.csv (336 rows), run_gate_v1_pinned_time.csv (3,150), inversion_v4.csv (6,600), inversion_v4_heldout_full.csv (7,020), inversion_v4_scale.csv (4,320), deletion_backbone_density_v1.csv (6,336), deletion_backbone_density_heldout_v1.csv (7,920), route-v2 counts/timing (540/11,340), preserve-v3 counts/timing (630/13,230), and rotate-v4 counts/timing (720/15,120). All rows passed output validation.